Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 3.3

Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

3. Паттерн Factory Method. Фигуры игры «тетрис». Реализовать процесс случайного выбора фигуры из конечного набора фигур. Предусмотреть появление супер-фигур с большим числом клеток, чем обычные.

Диаграмма классов

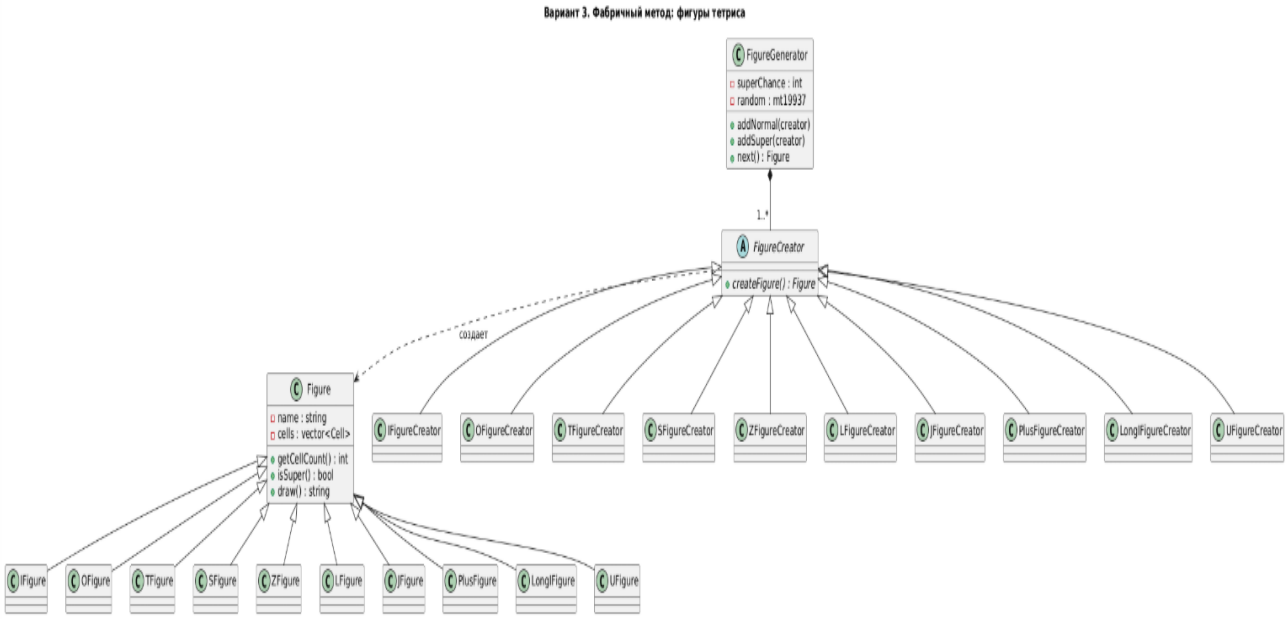
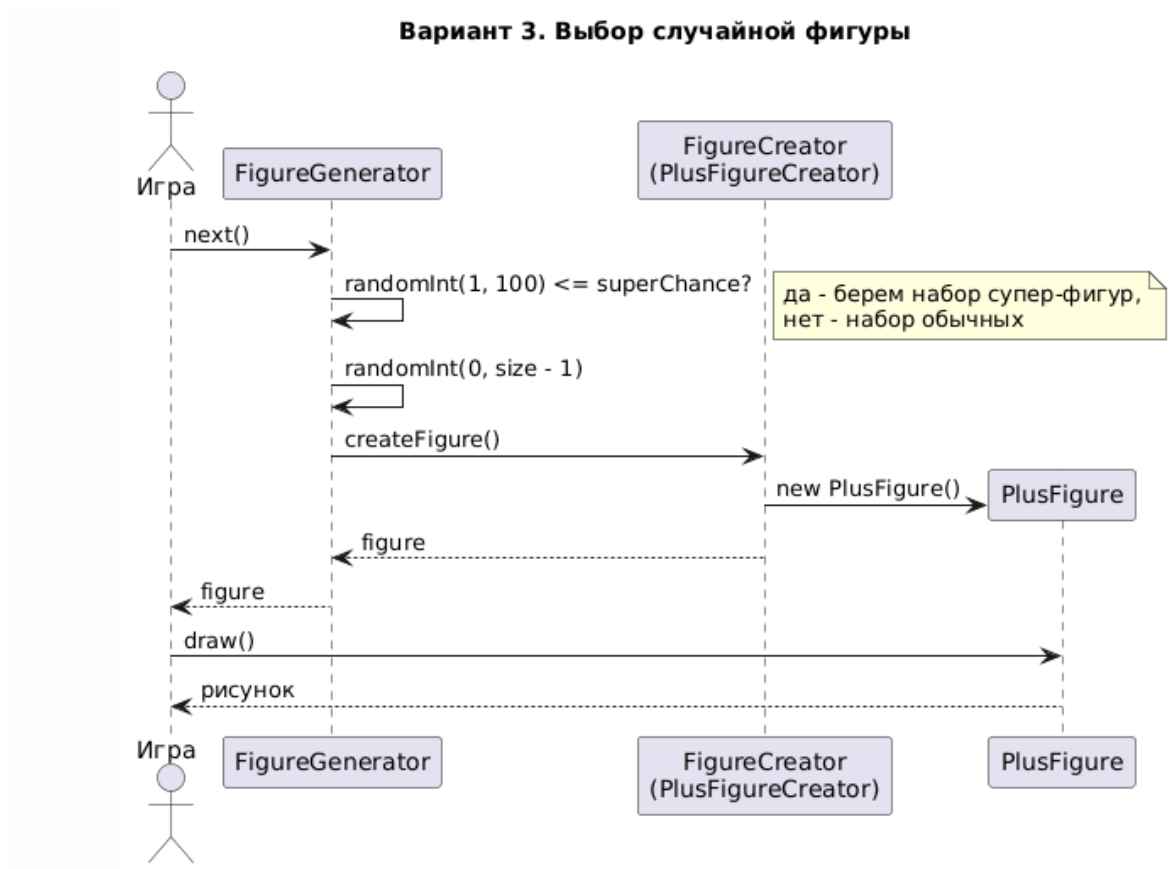


Диаграмма последовательности



Исходный код программы

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <random>
#include <set>
#include <clocale>

using namespace std;

// Клетка фигуры: столбец и строка
struct Cell {
   int x;
   int y;
};

// ======================== Продукт ========================
```

```
class Figure {
public:
   Figure(const string& name, const vector<Cell>& cells) : name(name), cells(cells) {}
   virtual ~Figure() {}

   string getName() const { return name; }
   int getCellCount() const { return (int)cells.size(); }
   const vector<Cell>& getCells() const { return cells; }

   // Обычные фигуры тетриса состоят из 4 клеток, супер-фигуры - из большего числа
   bool isSuper() const { return getCellCount() > 4; }

   // Рисунок фигуры символами '#'
   string draw() const {
      int width = 0, height = 0;
      for (const Cell& c : cells) {
         if (c.x + 1 > width) width = c.x + 1;
         if (c.y + 1 > height) height = c.y + 1;
      }
      vector<string> grid(height, string(width, '.'));
      for (const Cell& c : cells) grid[c.y][c.x] = '#';

      string result;
      for (const string& row : grid) result += "  " + row + "\n";
      return result;
   }

private:
   string name;
   vector<Cell> cells;
};

// ======================== Конкретные продукты ========================

// Обычные фигуры (4 клетки)
class IFigure : public Figure {
public:
   IFigure() : Figure("I", { {0, 0}, {1, 0}, {2, 0}, {3, 0} }) {}
};

class OFigure : public Figure {
public:
   OFigure() : Figure("O", { {0, 0}, {1, 0}, {0, 1}, {1, 1} }) {}
};

class TFigure : public Figure {
public:
```

```
    TFigure() : Figure("T", { {0, 0}, {1, 0}, {2, 0}, {1, 1} }) {}
};

class SFigure : public Figure {
public:
    SFigure() : Figure("S", { {1, 0}, {2, 0}, {0, 1}, {1, 1} }) {}
};

class ZFigure : public Figure {
public:
    ZFigure() : Figure("Z", { {0, 0}, {1, 0}, {1, 1}, {2, 1} }) {}
};

class LFigure : public Figure {
public:
    LFigure() : Figure("L", { {0, 0}, {0, 1}, {0, 2}, {1, 2} }) {}
};

class JFigure : public Figure {
public:
    JFigure() : Figure("J", { {1, 0}, {1, 1}, {1, 2}, {0, 2} }) {}
};

// Супер-фигуры (5 клеток)
class PlusFigure : public Figure {
public:
    PlusFigure() : Figure("Супер-Плюс", { {1, 0}, {0, 1}, {1, 1}, {2, 1}, {1, 2} }) {}
};

class LongIFigure : public Figure {
public:
    LongIFigure() : Figure("Супер-I", { {0, 0}, {1, 0}, {2, 0}, {3, 0}, {4, 0} }) {}
};

class UFigure : public Figure {
public:
    UFigure() : Figure("Супер-U", { {0, 0}, {2, 0}, {0, 1}, {1, 1}, {2, 1} }) {}
};

// ======================== Создатель ========================

// Объявляет фабричный метод createFigure(). Какую фигуру создавать,
// решают наследники.
class FigureCreator {
public:
    virtual ~FigureCreator() {}
    virtual unique_ptr<Figure> createFigure() const = 0;
```

```
};

// ======================== Конкретные создатели ========================

class IFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new IFigure()); }
};

class OFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new OFigure()); }
};

class TFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new TFigure()); }
};

class SFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new SFigure()); }
};

class ZFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new ZFigure()); }
};

class LFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new LFigure()); }
};

class JFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new JFigure()); }
};

class PlusFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new PlusFigure()); }
};

class LongIFigureCreator : public FigureCreator {
public:
   unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new LongIFigure()); }
```

```
};

class UFigureCreator : public FigureCreator {
public:
    unique_ptr<Figure> createFigure() const override { return unique_ptr<Figure>(new UFigure()); }
};

// ======================== Генератор фигур ========================

// Случайно выбирает создателя из конечного набора и вызывает его фабричный метод.
// С вероятностью superChance (в процентах) выбирается супер-фигура.
// Генератор не знает конкретных классов фигур: работает только с FigureCreator.
class FigureGenerator {
public:
    FigureGenerator(int superChance, unsigned int seed)
        : superChance(superChance), random(seed) {}

    void addNormal(unique_ptr<FigureCreator> creator) { normal.push_back(move(creator)); }
    void addSuper(unique_ptr<FigureCreator> creator) { super.push_back(move(creator)); }

    unique_ptr<Figure> next() {
        bool useSuper = !super.empty() && (normal.empty() || randomInt(1, 100) <= superChance);
        const vector<unique_ptr<FigureCreator>>& pool = useSuper ? super : normal;
        if (pool.empty()) return nullptr;
        int index = randomInt(0, (int)pool.size() - 1);
        return pool[index]->createFigure();
    }

private:
    int superChance;
    mt19937 random;
    vector<unique_ptr<FigureCreator>> normal;
    vector<unique_ptr<FigureCreator>> super;

    int randomInt(int from, int to) {
        uniform_int_distribution<int> dist(from, to);
        return dist(random);
    }
};

// Заполняет генератор стандартным набором фигур
void fillGenerator(FigureGenerator& generator) {
    generator.addNormal(unique_ptr<FigureCreator>(new IFigureCreator()));
    generator.addNormal(unique_ptr<FigureCreator>(new OFigureCreator()));
    generator.addNormal(unique_ptr<FigureCreator>(new TFigureCreator()));
    generator.addNormal(unique_ptr<FigureCreator>(new SFigureCreator()));
    generator.addNormal(unique_ptr<FigureCreator>(new ZFigureCreator()));
```

```
    generator.addNormal(unique_ptr<FigureCreator>(new LFigureCreator()));
    generator.addNormal(unique_ptr<FigureCreator>(new JFigureCreator()));
    generator.addSuper(unique_ptr<FigureCreator>(new PlusFigureCreator()));
    generator.addSuper(unique_ptr<FigureCreator>(new LongIFigureCreator()));
    generator.addSuper(unique_ptr<FigureCreator>(new UFigureCreator()));
}

// ======================== Тестирование ========================

class Tester {
public:
    Tester() : passed(0), failed(0) {}

    void check(bool condition, const string& name) {
        if (condition) {
            passed++;
            cout << "  [OK]   " << name << endl;
        } else {
            failed++;
            cout << "  [FAIL] " << name << endl;
        }
    }

    void printResult() const {
        cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
    }

private:
    int passed;
    int failed;
};

void testCreators(Tester& t) {
    TFigureCreator tCreator;
    unique_ptr<Figure> tFigure = tCreator.createFigure();
    t.check(tFigure->getName() == "T" && tFigure->getCellCount() == 4,
            "Создатель T создает фигуру T из 4 клеток");

    PlusFigureCreator plusCreator;
    unique_ptr<Figure> plus = plusCreator.createFigure();
    t.check(plus->getCellCount() == 5 && plus->isSuper(), "Супер-фигура больше обычной");

    OFigureCreator oCreator;
    t.check(oCreator.createFigure()->draw() == "  ##\n  ##\n", "Рисунок фигуры O");
}

void testNoSuper(Tester& t) {
```

```
    FigureGenerator generator(0, 1);
    fillGenerator(generator);
    bool anySuper = false;
    for (int i = 0; i < 200; i++) {
        if (generator.next()->isSuper()) anySuper = true;
    }
    t.check(!anySuper, "При шансе 0% супер-фигур нет");
}

void testOnlySuper(Tester& t) {
    FigureGenerator generator(100, 1);
    fillGenerator(generator);
    bool allSuper = true;
    for (int i = 0; i < 200; i++) {
        if (!generator.next()->isSuper()) allSuper = false;
    }
    t.check(allSuper, "При шансе 100% выпадают только супер-фигуры");
}

void testRandomChoice(Tester& t) {
    FigureGenerator generator(10, 42);
    fillGenerator(generator);
    set<string> names;
    int superCount = 0;
    for (int i = 0; i < 1000; i++) {
        unique_ptr<Figure> figure = generator.next();
        names.insert(figure->getName());
        if (figure->isSuper()) superCount++;
    }
    t.check(names.size() == 10, "За 1000 ходов выпали все 10 фигур");
    t.check(superCount > 0 && superCount < 300, "Супер-фигуры выпадают редко");
}

void testEmptyGenerator(Tester& t) {
    FigureGenerator generator(50, 1);
    t.check(generator.next() == nullptr, "Пустой генератор не создает фигур");
}

void runTests() {
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testCreators(t);
    testNoSuper(t);
    testOnlySuper(t);
    testRandomChoice(t);
    testEmptyGenerator(t);
    t.printResult();
```

```
}

// ======================== Демонстрация ========================

int main() {
    setlocale(LC_ALL, "Russian");

    random_device device;
    FigureGenerator generator(20, device());
    fillGenerator(generator);

    cout << "===== Демонстрация =====" << endl;
    cout << "Шанс супер-фигуры: 20%" << endl << endl;

    for (int i = 1; i <= 8; i++) {
        unique_ptr<Figure> figure = generator.next();
        cout << "Ход " << i << ": " << figure->getName() << " (клеток: " << figure->getCellCount()
             << (figure->isSuper() ? ", СУПЕР" : "") << ")" << endl;
        cout << figure->draw() << endl;
    }

    runTests();
    return 0;
}
```